CS6886W Assignment 1: VGG6 on CIFAR-10

Name / Roll no.: G S V RAMYA/ cs25m532

GitHub repository: https://github.com/cs25m532/vgg6-cifar10

W&B project: https://wandb.ai/cs25m532-iitm-india/vgg6-cifar10

Tools: PyTorch, Weights & Biases (W&B) sweeps, Google Colab with a Tesla T4 GPU.

Model: VGG6. It has four 3x3 conv layers (64, 64, 128, 128 channels), each followed by BatchNorm and the activation. A 2x2 max-pool comes after the 2nd and 4th conv. Then global average pooling and one linear layer. All runs use seed 42.

Question 1. Baseline

(a) Data preparation

The 50,000 training images are split into 45,000 for training and 5,000 for validation. The split uses a fixed seed, so all runs have the same validation images. The 10,000 test images are used only at the end for the test accuracy.

Training transforms (done on the GPU for each batch):

1. RandomCrop(32, padding=4)
2. RandomHorizontalFlip(p=0.5)
3. Normalize with mean (0.4914, 0.4822, 0.4465) and std (0.2023, 0.1994, 0.2010)
4. Cutout, one 16x16 hole

Validation and test images are only normalized with the same mean and standard deviation.

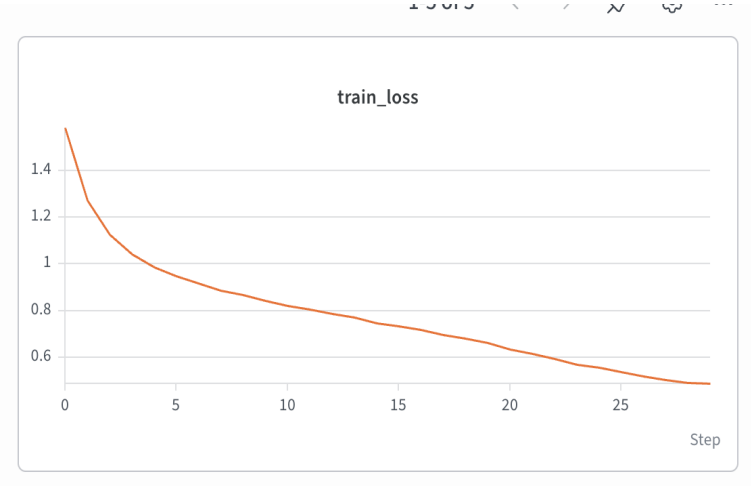
(b) Training one configuration

The best configuration from the experiments is used : ReLU, Nesterov SGD, learning rate 0.05, momentum 0.9, weight decay 5e-4, batch size 64, 30 epochs, cosine learning-rate schedule, BatchNorm.

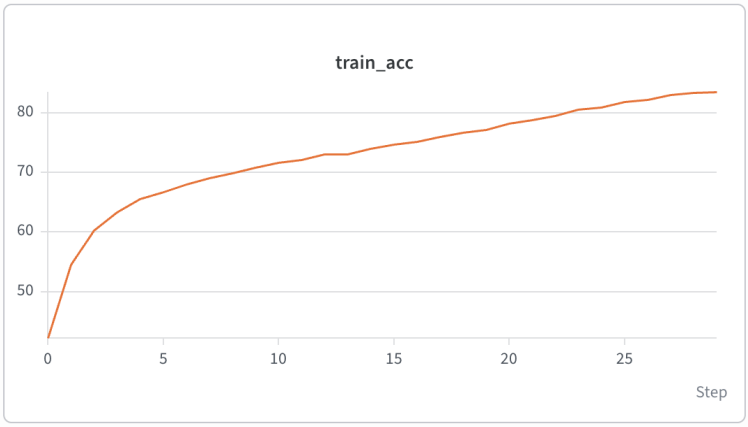
(c) Final accuracy and curves

Final test top-1 accuracy: **85.13%**. Best validation accuracy: 86.12% (epoch 30). Test loss: 0.443.

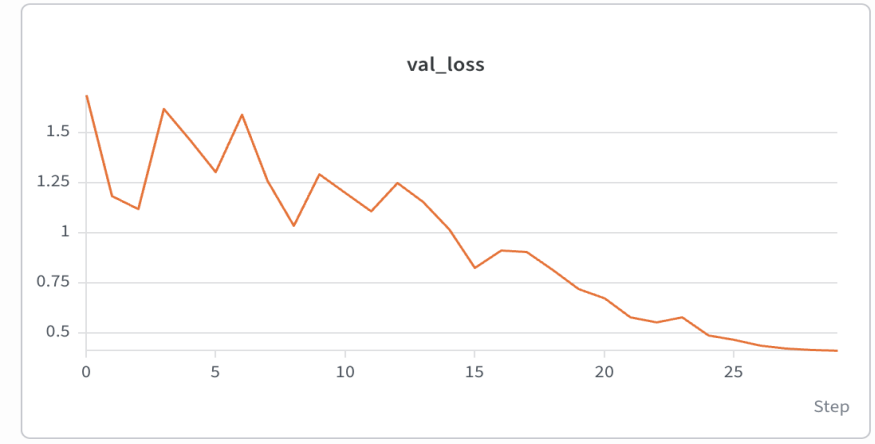
train loss:



train accuracy:



val loss:



val accuracy:



Question 2. Different configurations

All numbers are the best validation accuracy (%) of one run with seed 42. Runs were not repeated with other seeds, so differences of about 1 point may be noise. Two identical runs (ReLU, Nesterov SGD, lr 0.05, batch size 128, 20 epochs) gave exactly the same result (83.54 val, 82.61 test).

(a) Activation function

Nesterov SGD, lr 0.05, momentum 0.9, weight decay 5e-4, batch size 128, 20 epochs.

Activation	Val acc	Test acc
ReLU	83.54	82.61
GELU	83.04	82.54
SiLU	81.60	80.76
Tanh	72.68	71.78
Sigmoid	61.74	61.16

Sigmoid and Tanh are the worst. Both flatten out for large inputs, so gradients become small as they pass back through the layers. Sigmoid is also not centred at zero and its slope is at most 0.25, so it is the slowest. Tanh is centred and its slope goes up to 1, so it does better, but it is still about 10 points below ReLU after 20 epochs. BatchNorm helps both, but not enough to close the gap.

ReLU, GELU and SiLU are within about 2 points of each other. ReLU was best in this short run. GELU and SiLU did not show a gain, but the gaps are small and come from a single seed.

(b) Optimizer

ReLU, batch size 128, 15 epochs, momentum 0.9, weight decay 5e-4.

Optimizer	lr 0.001	lr 0.01	lr 0.1
SGD (momentum)	55.40	73.84	82.78
Nesterov SGD	55.28	74.28	83.14
Adam	79.86	77.02	36.78
Adagrad	54.00	72.82	79.82
RMSprop	75.42	55.06	36.66
Nadam	79.80	79.80	33.78

- SGD and Nesterov SGD need a large learning rate. They reach about 83% at lr 0.1 but only about 55% at 0.001, because 15 epochs is too short for such small steps. Nesterov is about 0.4 points ahead of plain momentum at 0.1 and 0.01, which is within noise.
- Adam and Nadam work the other way. They scale each parameter's step using running gradient statistics, so they reach about 80% at lr 0.001. At lr 0.1 they break down (around 35%). Nadam is Adam with Nesterov momentum and behaves very similarly. It handled lr 0.01 better (79.80 against 77.02).
- Adagrad adds up squared gradients, so its step size keeps shrinking. It needs a large starting rate (best at 0.1) and is slow at 0.001.
- RMSprop was good only at lr 0.001 and got much worse at larger rates. It also ran with momentum 0.9 here, which probably makes it less stable.
- The best result in the table is Nesterov SGD at lr 0.1. Adaptive optimizers need a much smaller learning rate to work at all, so matching the learning rate to the optimizer matters a lot. Weight decay is L2 for all of them.

(c) Batch size, epochs and learning rate

ReLU, Nesterov SGD, momentum 0.9, weight decay 5e-4.

Batch size	Epochs	lr 0.01	lr 0.05
64	10	74.26	80.66
64	20	79.40	84.36
64	30	82.50	86.12
128	10	69.76	78.20
128	20	76.28	83.54
128	30	79.60	85.96
256	10	64.82	75.36
256	20	72.54	80.98
256	30	75.76	84.02

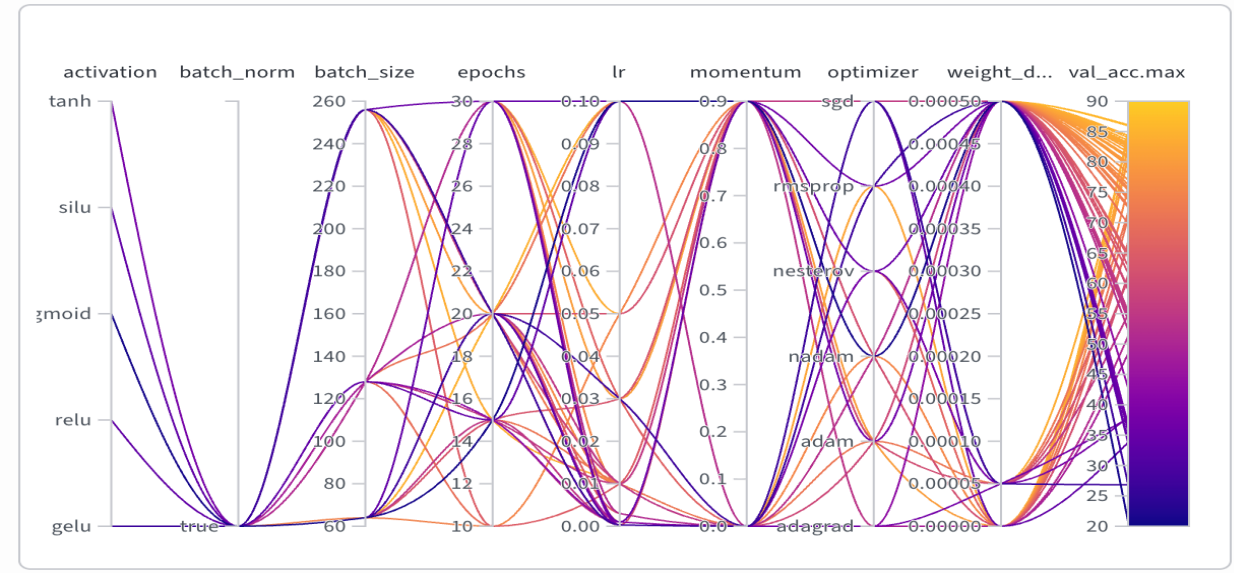
- More epochs helped in every case. Accuracy was still rising at 30 epochs, so the models are undertrained, not overfitting.

- lr 0.05 beat lr 0.01 by 4 to 11 points. With only 10 to 30 epochs, 0.01 is too slow.
- For the same number of epochs, smaller batches did better. At 30 epochs and lr 0.05 the results are 86.12, 85.96 and 84.02 for batch sizes 64, 128 and 256. A smaller batch gives more weight updates per epoch and noisier gradients. A larger batch probably needs a higher learning rate to catch up, but this was not tested.

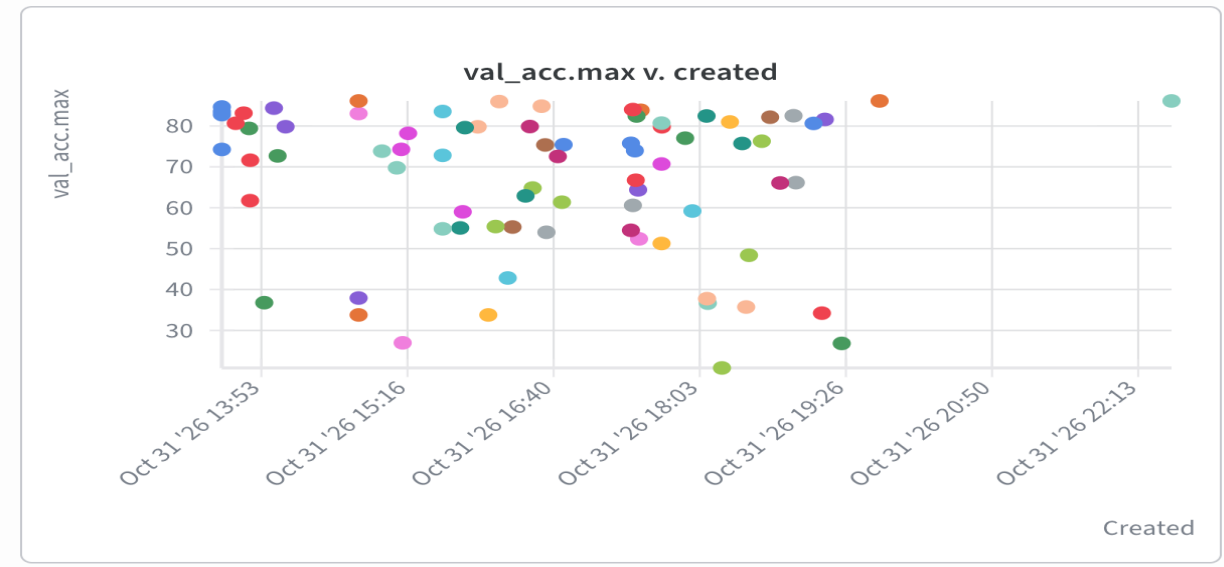
Question 3. Plots

All plots were made in W&B from all runs in the project.

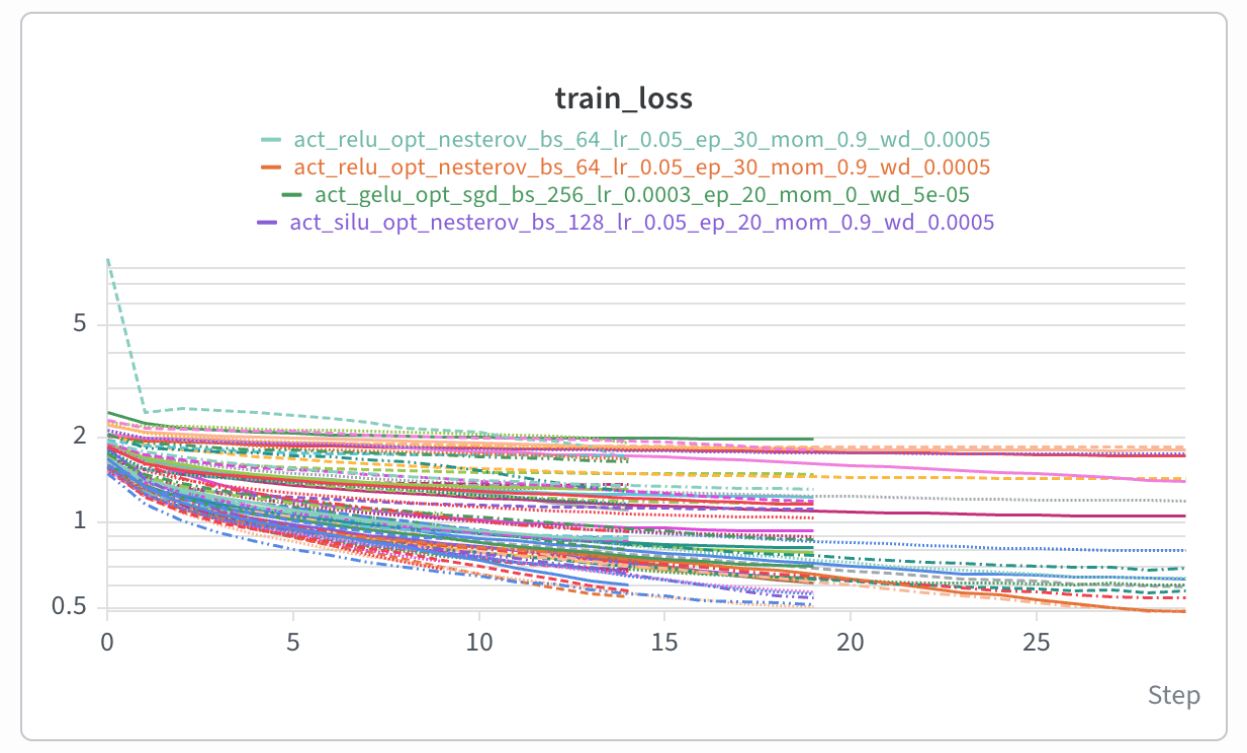
(a) Parallel-coordinates plot:



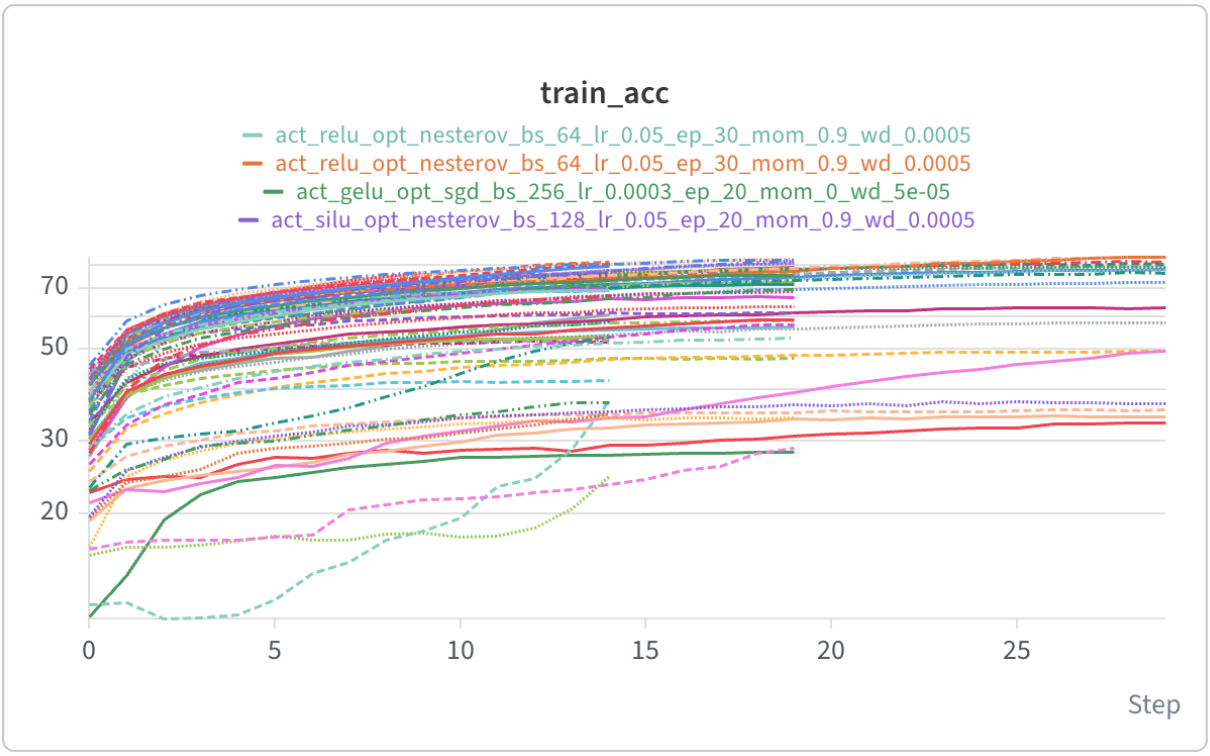
(b) Validation accuracy vs. step (scatter):



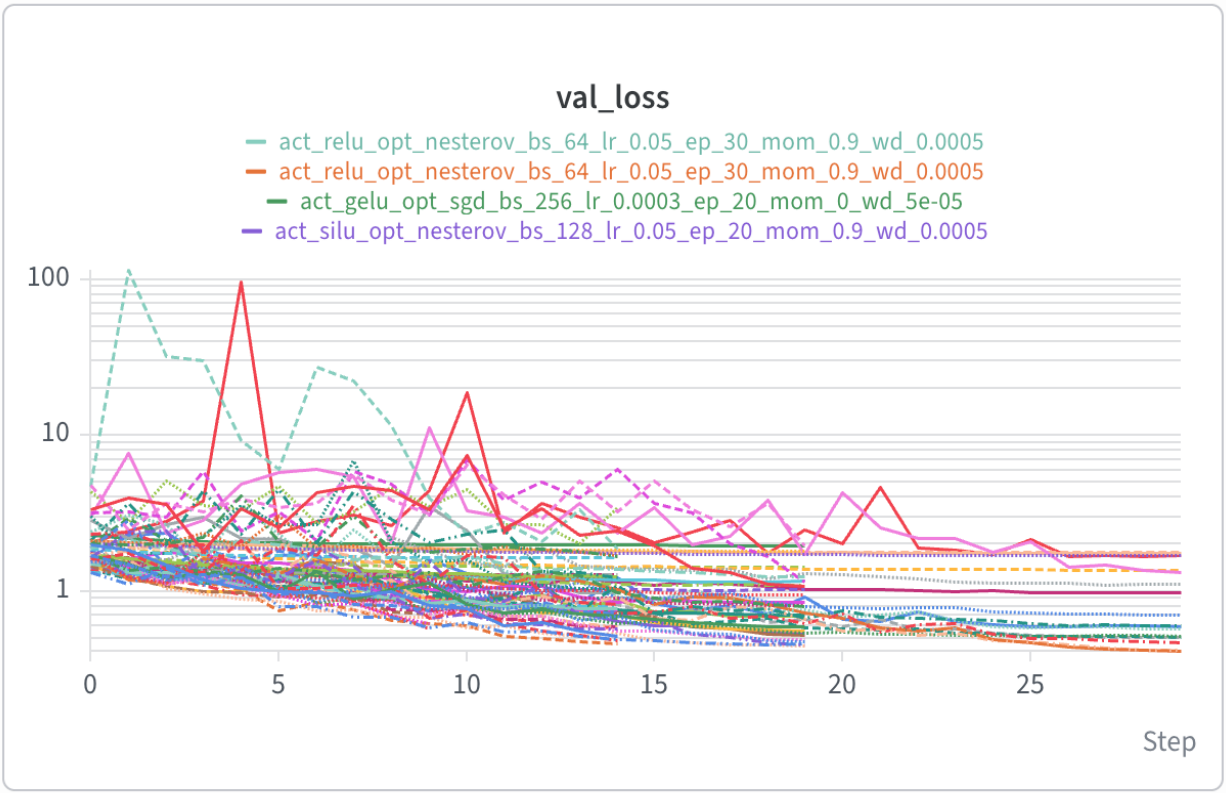
(c) Training loss:



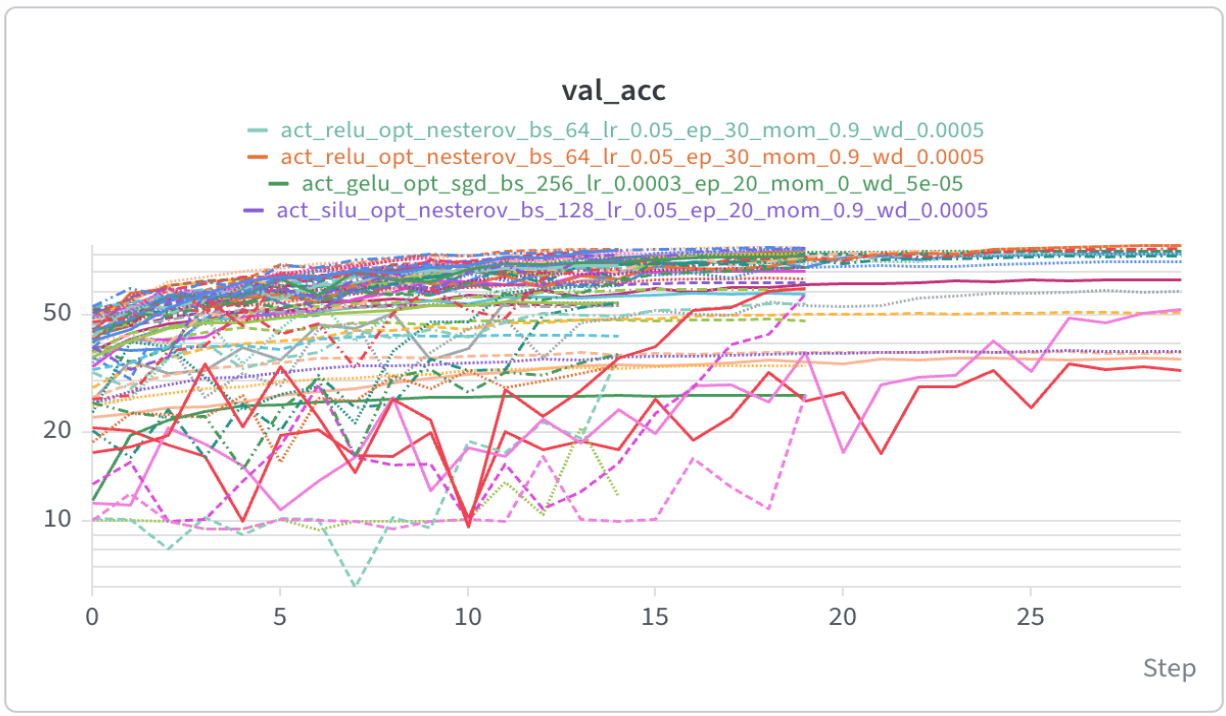
Training accuracy:



Validation loss:



Validation accuracy:



Four W&B sweeps were run: activation (5 runs), optimizer (18 runs), batch size / epochs / learning rate (18 runs) and a random search (about 35 runs). Colab disconnected a few times and some runs crashed. The sweeps did not repeat those configurations, so the crashed runs were deleted and 7 grid configurations were run by hand with the same arguments and seed. Those runs have no sweep ID in W&B.

Question 4. Final model

Best validation accuracy across all runs:

Parameter	Value
Activation	ReLU
Optimizer	Nesterov SGD
Learning rate	0.05
Momentum	0.9
Weight decay	0.0005
Batch size	64
Epochs	30
Scheduler	Cosine
Seed	42

Result: validation accuracy 86.12%, test accuracy 85.13%. This configuration was run twice in separate Colab sessions. Both runs gave the same accuracy, and the two saved models have identical weights.

Question 5. Code and reproducibility

(a) The code is split into separate files:

- data.py (data and augmentation)
- model.py (VGG6)
- train.py (training and W&B logging)
- evaluate.py (evaluation)
- utils.py (seeding, optimizers, scheduler)
- sweeps/*.yaml (W&B sweep configs)
- export_results.py.

(b) The README has the exact commands, the environment and dependency versions (torch 2.11.0, torchvision 0.26.0, numpy 2.1.3, pandas 2.2.3, wandb 0.28.1) and the seed settings (seed 42, deterministic cuDNN).

(c) The trained model is in the repository as models/best_model.pt. Repository: https://github.com/cs25m532/vgg6-cifar10